

CCSDS/ECSS TM Pseudo-Randomizer: What, Where, and Why (with TX/RX Placement)

Joshua Wille

October 29, 2025

Abstract

This note summarizes the CCSDS/ECSS telemetry (TM) pseudo-randomizer: the LFSR sequence, its reset rule, and the exact placement in the transmit (TX) and receive (RX) chains for common coding options (uncoded, convolutional, Reed–Solomon, concatenated, turbo). Practical checklists and implementation tips are included, along with primary standard references.

1 Essentials

Purpose. The TM pseudo-randomizer whitens the bitstream to aid receiver acquisition, reduce spectral lines, and improve clock/data recovery.

Mechanism. A multiplicative scrambler via an 8-bit LFSR, exclusive-OR'd bitwise with the data payload (never with the ASM).

$$h(x) = x^8 + x^7 + x^5 + x^3 + 1 \quad (1)$$

$$\text{Period} = 2^8 - 1 = 255 \text{ bits} \quad (2)$$

$$\text{Reset (seed)} = \text{all ones (0xFF) at the start of each unit} \quad (3)$$

Reset rule. Re-initialize the LFSR to all ones at the start of *each* transfer frame or codeblock, depending on where the randomizer is applied (details in §2). The *first* PRBS bit XORs the *first* payload bit.

2 Where it Lives in the Chain

The standards place the pseudo-randomizer on the *payload that will be protected as a unit*, and it *never* touches the Attached Sync Marker (ASM). After randomization, the ASM is appended (and if convolutional coding is used, the ASM is included in that convolutional encoding step).

Canonical TX Orders (by coding option)

Let TF denote the TM Transfer Frame. Let PRBS8 denote XOR with the LFSR sequence.

1. Uncoded (no FEC):

$$\text{TF} \rightarrow \text{PRBS8} \rightarrow \text{ASM} \rightarrow \text{modulate}$$

2. Convolutional only:

$$\text{TF} \rightarrow \text{PRBS8} \rightarrow \text{ASM} \rightarrow \text{Conv (K=7, r=1/2)} \rightarrow \text{modulate}$$

3. Reed–Solomon only (with interleave depth I):

TF $\rightarrow$ RS(255,223) $\xrightarrow{\text{interleave } I}$ PRBS8 (on RS codeblock) $\rightarrow$ ASM $\rightarrow$ modulate

4. Concatenated (RS + Convolutional) (*classic CCSDS*):

TF $\rightarrow$ RS(255,223) $\xrightarrow{\text{interleave } I}$ PRBS8 (on RS codeblock) $\rightarrow$ ASM $\rightarrow$ Conv $\rightarrow$ modulate

5. Turbo:

TF $\rightarrow$ Turbo encode $\rightarrow$ PRBS8 (on turbo codeblock) $\rightarrow$ ASM $\rightarrow$ modulate

Key rule. *Do not XOR the ASM with the PRBS.* The ASM must pass through the channel unaltered.

RX Order (inverse)

1. **Frame sync on ASM**, acquire CADU boundaries.
2. If present: **Viterbi decode** (Conv) $\rightarrow$ recovered CADU payload + ASM removed.
3. **Derandomize** (apply PRBS8 again, same reset rule) on the recovered *payload unit*:
 - TF if uncoded/conv-only,
 - RS codeblock if RS/concatenated,
 - turbo codeblock if turbo.
4. **Block decode**: RS deinterleave + decode (or turbo decode).
5. Recover **TF** (1115 B typical for $I=5$ with RS(255,223)).

3 Lengths and Tagging (RS+Conv example)

For the classic CCSDS TM with RS and conv:

$$\text{TF (randomized later at block-level)} = 1115 \text{ B} = 5 \times 223 \quad (4)$$

$$\text{After RS(255,223) with } I=5 = 1275 \text{ B} = 5 \times 255 \quad (5)$$

$$\text{CADU (add 4-B ASM)} = 1279 \text{ B} \quad (6)$$

$$\text{After Conv (r=1/2)} = 2 \times 1279 \times 8 \text{ bits} \quad (7)$$

Tip: Maintain consistent stream tags across stages: $1115 \rightarrow 1275 \rightarrow 1279 \rightarrow$ (post-conv bits).

4 Implementation Notes (Gotchas)

1. **Bit alignment.** The first PRBS bit XORs the first data bit of the chosen unit (TF or codeblock). Ensure consistent bit endianness when pack/unpack around XOR.
2. **Reset every unit.** Re-seed LFSR to 0xFF at each TF/codeblock boundary. *Do not* allow the sequence to run across frame boundaries.
3. **ASM untouched.** Never randomize or derandomize the ASM.
4. **RS virtual fill.** If you use shortened RS via virtual fill, those leading zeros are logical (not transmitted) and are handled internal to RS; the randomizer should never “see” them.
5. **Conv placement.** With convolutional coding, insert ASM *before* the conv encoder, so the ASM is also convolutionally encoded (but still not randomized).

5 Reference PRBS and Unit Tests

The standard gives a reference start-up sequence (first several PRBS bits) for the all-ones seed. Use it to sanity-check your implementation:

1. Reset LFSR to 0xFF.
2. Generate the first 40 PRBS bits and compare with the reference sequence from the spec.
3. XOR a known test payload and confirm TX → RX loopback recovers the original.

6 Minimal LFSR Pseudocode

```
uint8_t lfsr = 0xFF; // reset per unit (TF or codeblock)
uint8_t prbs_bit() {
    // taps for  $x^8 + x^7 + x^5 + x^3 + 1$  (feedback from MSB before shift)
    uint8_t fb = ((lfsr >> 7) ^ (lfsr >> 6) ^ (lfsr >> 4) ^ (lfsr >> 2)) & 0x01;
    uint8_t out = (lfsr >> 7) & 0x01; // output bit
    lfsr = ((lfsr << 1) | fb) & 0xFF; // shift left, insert feedback into LSB
    return out;
}
// For each data bit d: d_scrambled = d ^ prbs_bit();
```

7 Quick Checklists

TX sanity

- Chosen coding option matches a TX order in §2.
- Randomizer placed on TF (uncoded/conv) or *on codeblock* (RS/turbo).
- ASM appended *after* randomization; never randomized.
- With convolutional coding, ASM is inserted *before* conv encode.

RX sanity

- Frame sync on ASM; strip ASM region from the payload unit boundary.
- If present, Viterbi first; *then* derandomize on the recovered payload unit.
- Then block decode (RS/turbo) to recover the TF.

8 Why Not Randomize Before RS When Using RS?

For block codes, the randomizer is defined on the *block* that is transmitted as a unit (the codeblock). Applying PRBS at the codeblock level preserves the assumptions behind interleaving/decoding and ensures the ASM remains literal and detectable.

9 Sources (Primary Standards and Notes)

- [1] **CCSDS 131.0-B-3** (TM Synchronization and Channel Coding). Draft copy (2019-10-16) is included in your project folder: “*CCSDS-131.0-B-3_forECSS(Draft-16Oct2019).pdf*”. Defines PRBS polynomial, reset-to-all-ones, and TX/RX placement rules (ASM not randomized).

- [2] **ECSS-E-ST-50-01C** (31 July 2008), *Space engineering – Communications*. ECSS adoption/overview of CCSDS TM sync & channel coding, including block diagrams showing pseudo-randomizer placement relative to ASM and convolutional encoder.
- [3] NASA/CCSDS guidance slide decks summarizing “*Randomizer for High Data Rates*” (commonly reprinted in mission integration notes): rule of thumb—TF-level for uncoded/conv; *block-level* (codeblock) for RS/turbo; ASM never randomized.

One-page TL;DR

- LFSR: $x^8+x^7+x^5+x^3+1$, period 255, reset=0xFF per unit.
- Randomize *payload only*: TF (uncoded/conv) or codeblock (RS/turbo).
- Append ASM *after* randomization; never XOR the ASM.
- If conv is used: ASM goes *into* conv encoder (still not randomized).
- RX: ASM sync → (Viterbi) → derandomize unit → block decode.